

I Know What Would Break at *JPMC / PayPal* Scale. Here's the Redesign.

My Western Union pipeline handled 10M events/day reliably. But I've thought hard about where it fails at 10×, 100×, and 1000× that load — and how I'd rebuild each layer to survive it.

WHAT I BUILT	PAYPAL TIER	JPMC TIER
115 /s 10M events/day <1s P99 latency 3 stream types Single region	~1,500 /s ~40M txns/day + aux events <300ms P99 scoring 10+ stream types Multi-region active-active	50,000+ /s \$10T+ annually <50ms P99 on card rails Microsecond SLAs (ACH/RTGS) Global, multi-datacenter

Let Me Be Direct About Where My System Sits

The Western Union pipeline works. It processes 10M events/day at sub-second latency, catches fraud before settlement, and has been reliable in production. I'm proud of it.

It would also start visibly struggling somewhere around PayPal's volume, and fall over well before JPMC's. Not because the architecture is wrong — it isn't — but because every design embeds

assumptions about scale, and mine embeds assumptions that stop holding at 1,500 events/second.

This piece is my honest analysis of exactly what breaks, at what point, and what I'd replace. I find these thought experiments genuinely useful — they force you to understand why your current design works, not just that it does.

The Five Things That Actually Break

"Just add more Kafka partitions" is the lazy answer to scale questions. The things that genuinely fail under pressure are subtler:

LATENCY

At 115 events/second, 800ms P99 is fine. At 50,000/second, tail latency compounds — one slow ML scoring call backs up the enrichment layer. You need **per-stage latency budgets with hard enforcement**, not a single end-to-end SLA. The budget model I'd use: 2ms ingest, 12ms enrichment, 18ms scoring, 5ms action dispatch. Everything else is async. Total: under 50ms P99.

STATE SIZE

Flink's RocksDB state backend starts degrading around 100GB per task manager. At PayPal scale, checkpoint sizes balloon and recovery after a crash takes 10+ minutes — unacceptable when you're blocking live transactions. I'd move to three-tier state: hot context in JVM heap, 24-hour history in RocksDB, anything older offloaded to Redis Cluster or Aerospike.

REGULATORY

At JPMC, every fraud decision must be explainable to OCC examiners — not just logged but fully reconstructable. "The model scored it 0.87" is not a compliant explanation. You need SHAP values computed for every scored event, stored alongside the model version and input features in an immutable audit log. This is regulatory infrastructure, not a nice-to-have you add later.

GEO-DISTRIBUTION

Single-region active-passive fails at JPMC scale. A region outage that takes 30–45 seconds to failover costs millions on live payment rails. You need active-active multi-region from the start, with conflict resolution for shared fraud state — blacklists, velocity counters — designed as CRDTs or backed by a globally replicated store like CockroachDB.

MODEL FRESHNESS

Fraud patterns shift within hours during an active attack. A model trained yesterday may be blind to today's mule-network pattern. At tier-1 scale you need online learning or continuous retraining pipelines pushing model updates to scoring infrastructure in minutes, not deployment cycles.

The Redesigned Architecture



What I'd Actually Change, and Why

INGESTION: SCHEMA ENFORCEMENT BEFORE EVENTS TOUCH KAFKA

In the Western Union system, schema validation happened inside the pipeline — which meant a malformed event from a new producer caused a Flink job failure, not a clean rejection at source. At 115 events/second that's manageable. At 50,000/second, a bad producer firing at full rate for 30 seconds before someone notices can corrupt hours of checkpoint state.

I'd add a Confluent Schema Registry enforcing Avro contracts at the producer level. Schema-incompatible events get rejected before

they touch Kafka. The failure surface shrinks dramatically: producers fail fast and loudly, pipelines stay clean. I'd also change the partition key from `user_id` alone to `user_id + instrument_id` — at PayPal scale, a single user may have 10+ cards, and per-instrument velocity tracking across partitions requires expensive coordination you can avoid with a compound key.

STATE: THREE TIERS, NOT ONE

This is the one I'd be most worried about at PayPal volume. Flink's RocksDB state backend is excellent, but checkpoint sizes grow with state volume. Above roughly 100GB per task manager, recovery from failure starts taking 10+ minutes — which means that every time you lose a task manager in production, you're blocking transaction scoring for that partition for over ten minutes. That's unacceptable.

The fix is explicit state tiering: the last 60 seconds of per-user session context in JVM heap (microsecond access), the last 24 hours in RocksDB (millisecond), anything older in Redis Cluster or Aerospike (low-millisecond remote). The enrichment logic is rewritten to pull from the correct tier based on the time horizon of the feature being computed. The operational complexity is real — three stores instead of one — but the recovery time profile is completely different.

A FEATURE STORE: DECOUPLING FEATURES FROM THE PIPELINE

In the original design, features were computed inside the Flink job. That works, but it creates a hidden coupling that eventually becomes a serious problem: the features used to train your ML models offline are not guaranteed to be identical to the features computed at serving time. That gap — training/serving skew — is

one of the most common and hardest-to-diagnose causes of model degradation in production.

A dedicated feature store (Feast or Tecton) solves this. Features are computed once, stored consistently, and consumed identically by both the training pipeline and the serving path. This sounds like overhead when you're at Western Union scale. At PayPal scale, where you have dozens of models trained on hundreds of features, it's load-bearing infrastructure.

GRAPH ENGINE: SEEING THE NETWORK, NOT JUST THE TRANSACTION

Mule networks — coordinated fraud rings where money passes through chains of accounts before exiting the system — are invisible to per-transaction scoring. Each individual transaction looks clean. The pattern only emerges when you look at the account relationship graph.

```
// 3-hop mule chain detection at scoring time (Gremlin)
g.V().has('accountId',
txn.senderId) .repeat(out('transferred_to').simplePath()) .times(3) .where(has('flagged'
true)) .count() .next() // → ring_proximity_score fed into ensemble
```

At JPMC scale this graph has billions of edges. You can't query it naively at scoring time. The answer is pre-computing graph features — centrality scores, community membership, shortest path to a known-bad node — on a batch schedule and caching them in the feature store. The real-time query then just does a feature lookup, not a live graph traversal.

SCORING: ENSEMBLE OVER SINGLE MODEL, SHAP FOR EVERY DECISION

No single model dominates across all fraud typologies. A gradient-boosted tree catches velocity-based patterns well. A neural network handles device fingerprint anomalies better. A rule engine fires instantly on known signatures with zero model latency. I'd run all three in parallel and combine their outputs through a meta-scorer weighted by recent precision per fraud category.

On explainability: SHAP values need to be computed for every single scored event, not sampled. At JPMC, a customer disputing a blocked transaction triggers a regulatory process where the explanation must be deterministically reconstructable from logged inputs and the model version that was live at decision time. Sampling SHAP is not compliant. I'd store the full feature vector, the model version hash, the SHAP values, and the final score in an append-only S3 log with WORM retention.

At this scale, the difference between 94% and 96% precision isn't two percentage points. It's tens of millions of dollars in avoided false-positive friction annually. That's why ensemble scoring and continuous model evaluation aren't luxuries — they're economics.

THE LATENCY BUDGET MODEL

The most important discipline change at tier-1 is replacing a single end-to-end SLA with owned per-stage budgets, enforced with circuit breakers at every boundary:

STAGE	BUDGET	ENFORCEMENT
Kafka ingest	2ms P99	linger.ms=0, acks=1 for low-value streams
Flink enrichment	12ms P99	Tiered state, async graph lookup, watermark tuning
Ensemble scoring	18ms P99	Parallel model calls, circuit breaker at 15ms
Action dispatch	5ms P99	Pre-warmed pools, local decision cache
Audit write	async	Fire-and-forget to S3 — never on the critical path
Total P99	<50ms	Jaeger end-to-end tracing, SLO alert at 40ms

The audit write being off the critical path is the detail that surprises people. At 50,000 events/second, a synchronous 5ms audit write contributes 250 seconds of cumulative blocking per second of throughput. Everything that isn't the fraud decision itself gets pushed async.

The Honest Bit: What My Current Design Gets Right

I want to be clear that this isn't a critique of the Western Union architecture. The three-layer topology, entity-keyed partitioning, async ML integration, externalised rules, and dead-letter queues are all correct and would carry forward unchanged into a tier-1 system. The original design makes exactly the right bets for its scale.

The upgrades I've described — feature store, graph engine, ensemble scoring, tiered state, SHAP logging — are additive, not

corrective. None of them make sense below a certain volume threshold. A feature store at 115 events/second is engineering overhead you don't need. At 1,500 events/second with a dozen ML models, it's the only way to keep training and serving aligned.

So why write this if I haven't built it yet?

Because I think the gap between "built something that works" and "understands what comes next" is where engineering judgment actually lives. Anyone can run a system. **Knowing precisely where it breaks, and why, and what the correct fix is** — that's the work. This is my thinking made visible. If it's wrong somewhere, I'd genuinely like to know.